

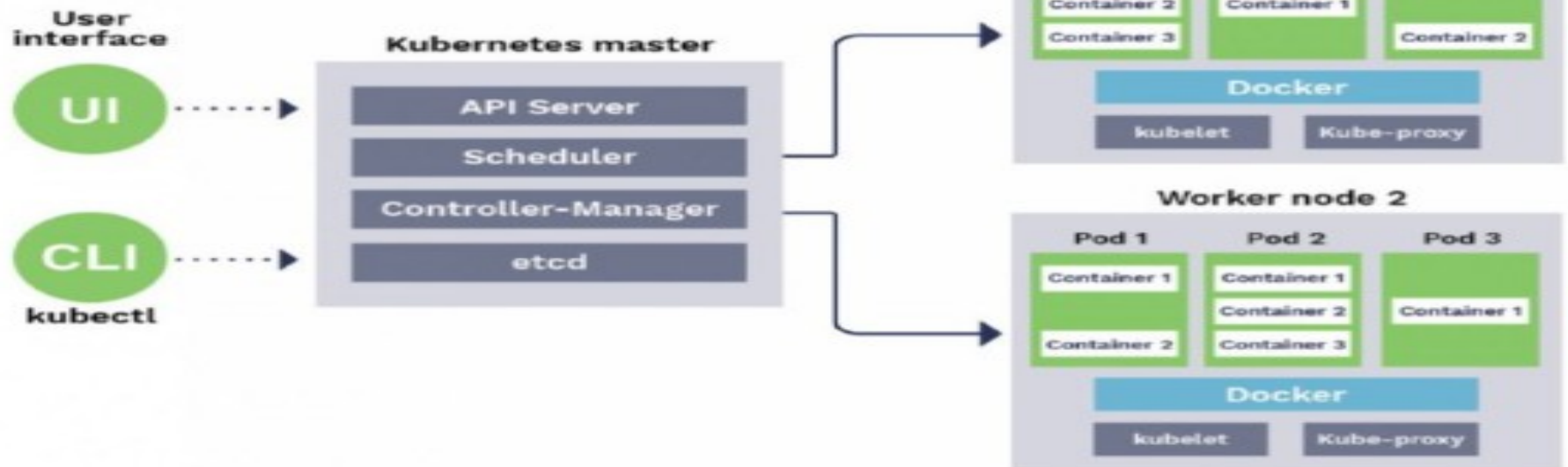


Kubernetes servelet et service mesh

Réalisé et soutenu par :
Magnidé G. Natus FAGLA
97170520

Architecture du système orchestrateur kubernetes

Kubernetes architecture



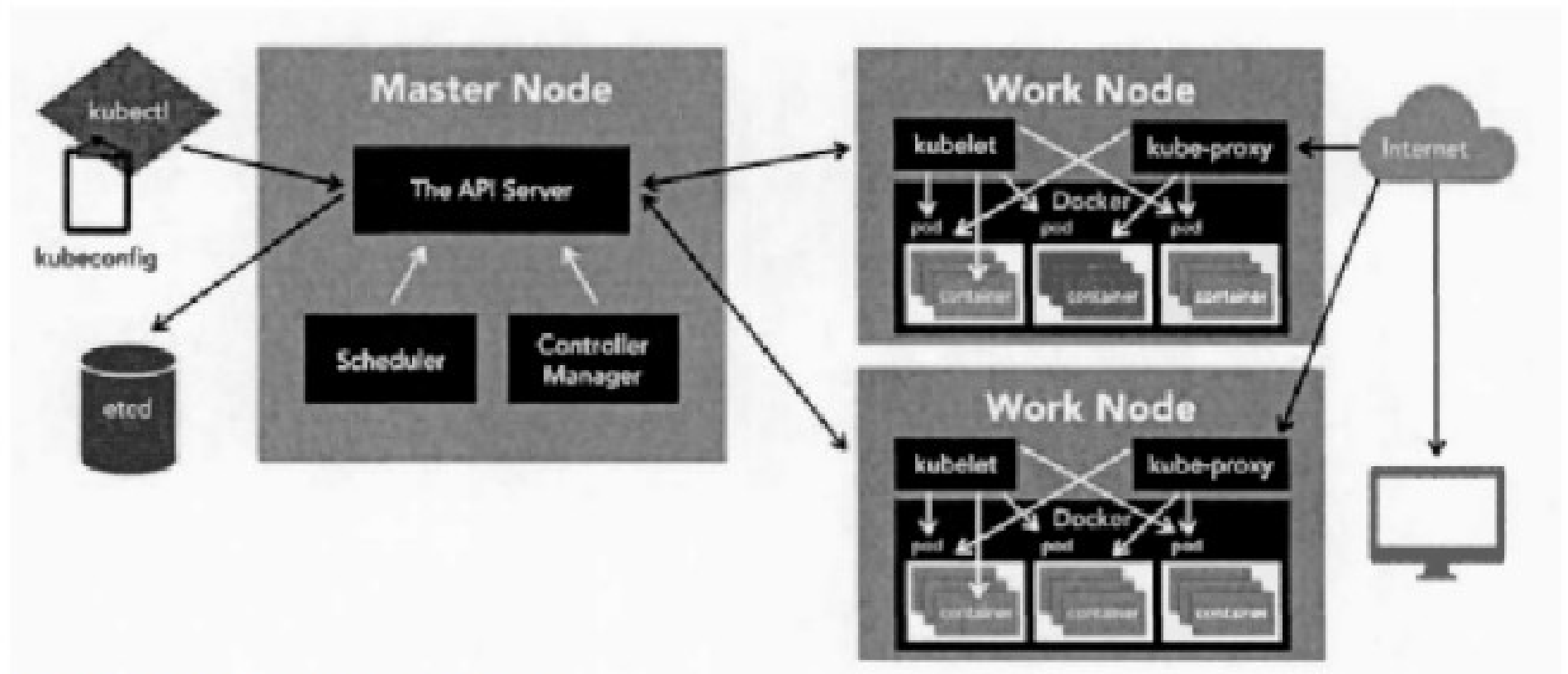


fig : Architecture système kubernetes

Nœud Maître (master node)

Le Nœud Maître (Cluster Master) a un nombre vraiment ridicule d'alias. Il est également appelé nœud maître, maître Kubernetes, plan de contrôle du cluster, plan de contrôle ou encore simplement maître.

Peu importe comment vous l'appellez, il dirige les nœuds de travail. Les maîtres prennent des décisions de planification, répondent aux événements, implémentent les modifications et surveillent le cluster. Les nœuds de travail et le maître ont des composants de sous-processus.

► Les principaux processus systeme de master node

- **API SERVER**

Le serveur d'API Kubernetes valide et configure les données pour les objets API qui incluent les pods, les services, les contrôleurs de réplication et autres. Le serveur API dessert les opérations REST et fournit l'interface à l'état partagé du cluster à travers lequel tous les autres composants interagissent.

C'est l'interface pour le contrôle de Kubernetes. (aka. kube-apiserver) pensez à un hub.

► Les principaux processus systeme de master node

- Controller-manager

Dans Kubernetes, un contrôleur est une boucle de contrôle qui surveille l'état partagé du cluster via l'apiserver et apporte des modifications en tentant de déplacer l'état actuel vers l'état souhaité. Des exemples de contrôleurs livrés avec Kubernetes aujourd'hui sont : le contrôleur de réplication, le contrôleur de points de terminaison, le contrôleur d'espace de noms et le contrôleur de comptes de service.

Processus qui exécute des contrôleurs pour gérer les tâches d'arrière-plan du cluster. Pensez à un contrôleur de cluster .

Contrôle des itération de toutes sortes

- **Scheduler**

Le planificateur Kubernetes est un processus de plan de contrôle qui attribue des pods à des nœuds. Le planificateur détermine quels nœuds sont des emplacements valides pour chaque pod dans la file d'attente de planification en fonction des contraintes et des ressources disponibles. Le planificateur classe ensuite chaque nœud valide et lie le pod à un nœud approprié. Plusieurs planificateurs différents peuvent être utilisés dans un cluster ; kube-scheduler est l'implémentation de référence

Etcd

Distributeur de stockage clé-valeur pour les données d'état du cluster. Pensez aux informations sur le cluster .

Etcd est comme un magasin de valeurs clés cohérent et hautement disponible utilisé comme magasin de sauvegarde de Kubernetes pour toutes les données du cluster.

Si votre cluster Kubernetes utilise etcd comme magasin de sauvegarde, assurez-vous d'avoir un plan de sauvegarde pour ces données.

► Les principaux processus système du worker node

- Container runtime
- Télécharge des images et exécute des conteneurs. Par exemple, Docker est un conteneur Runtime. Pensez au conteneur Docker.

Un nœud de travail (worker node)

est également appelé nœud en abrégé. Un nœud est une abstraction

pour une machine, qu'il s'agisse d'une machine physique ou d'une machine virtuelle. Considérez un

nœud comme un serveur informatique. Un ou plusieurs pods s'exécutent sur un seul nœud de travail. Un

pod n'est jamais divisé entre deux nœuds. le contenu d'un pod est toujours situé et programmé ensemble

sur le même nœud. Qui dit aux nœuds de travail quoi faire ? Le maître

- Kubelet

Le kubelet est le principal "agent de nœud" qui s'exécute sur chaque nœud. Il peut enregistrer le nœud auprès de l'apiserver en utilisant l'un des éléments suivants : le nom d'hôte ; un indicateur pour remplacer le nom d'hôte ; ou une logique spécifique pour un fournisseur de cloud.

- Kube-proxy

Achemine les connexions vers les pods appropriés. Effectue également l'équilibrage de charge entre les pods pour un service. Pensez agent de la circulation.

Parcerelle reseaux entre le nœud worker et le nœud master

► Kubectl

kubectl appelé "kube contrôle " est l' outil de ligne de commande pour les K8s.

Comment cela fonctionne ? :

Une commande kubectl exécute une action telle que get, create ou describe

L'action est effectuée sur une ressource telle qu'un deployment, un StatefulSet ou un service.

Les commandes kubectl suivent ce format :

Kubectl [action] [ressource] [nom_ressource] [flags] ??

Les flags sont facultatifs dans de nombreux cas.

Par exemple, pour obtenir des informations sur tous les pods existants, vous pouvez exécuter cette commande courante : kubectl get pods.

NB : Si vous voulez éviter de taper kubectl un million de fois, créez un alias Bash. Avec alias k='kubectl' vous pouvez simplement taper kau lieu de kubectl.

Introduction

Nous observons de nos jours la non coordination entre l'équipe développeur et l'équipe des opérations qui gère l'environnement de production , De plus les déploiements d'application pose souvent un problème d'environnement d'une part de disponibilité, d'évolutivité, de résilience de tolérance aux panne d'autre part c'est fort ce cela nous avons proposé la mise en place d'une infrastructure DevOps basée sur Kubernetes

► Les principaux objets de kubernetes :

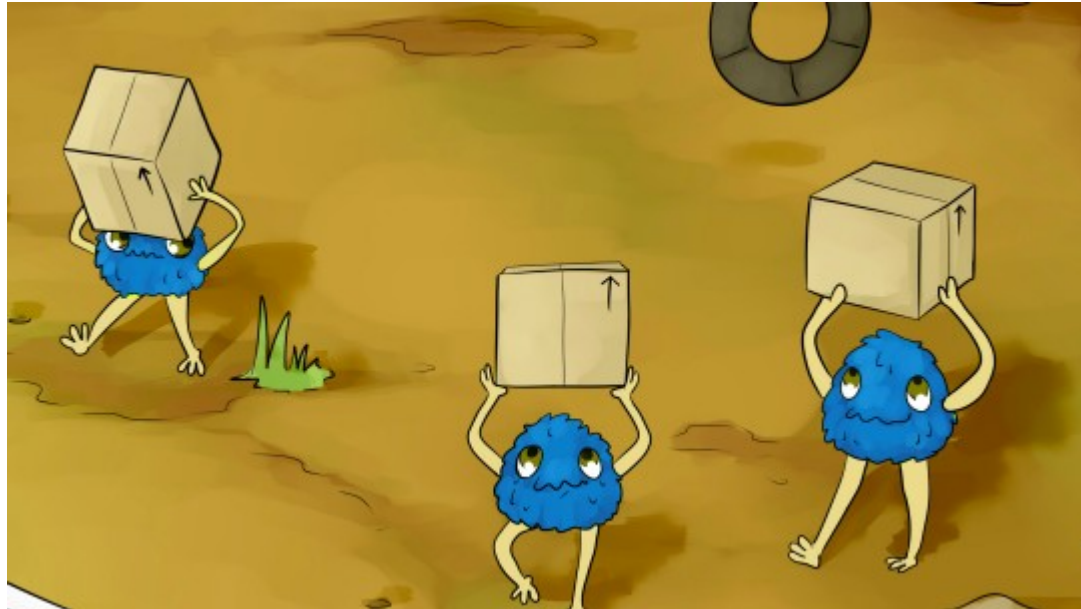
- **POD**

Le pod est l'unité basique pour demararer un conteneur dans kubernetes

Le Pod est le bloc de construction de base de Kubernetes. Un pod contient un groupe d'un ou plusieurs conteneurs. Généralement, chaque Pod a un conteneur.

Les Pods sont éphémères. Ils sont destinés à être redémarrés automatiquement lorsqu'ils meurent

illustration



- Deployment

Un déploiement est une abstraction d'ordre supérieur qui contrôle le déploiement et la maintenance d'un ensemble de pods. Derrière les scènes, il utilise un ReplicaSet pour faire fonctionner les pods, mais il offre une logique sophistiquée pour le déploiement, mettre à jour et mettre à l'échelle un ensemble de pods au sein d'un cluster.

illustration



- ReplicaSet

Un ReplicaSet garantit qu'un ensemble de pods configurés de manière identique s'exécutent au nombre de répliques souhaité. Si un module tombe, le ReplicaSet en met un nouveau en ligne en remplacement.

ilustration



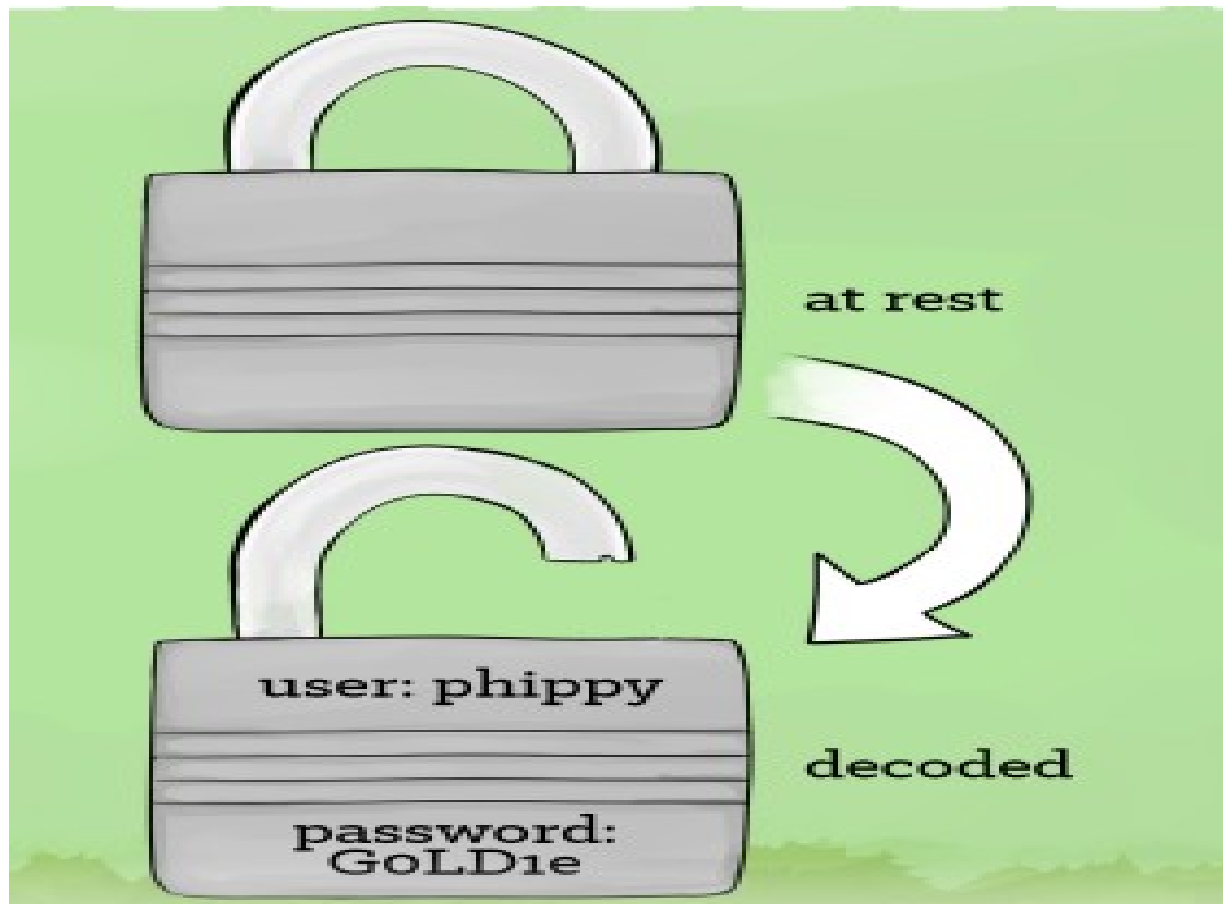
- Secret

Les secrets sont utilisés pour stocker des informations non publiques, telles que des jetons, des certificats ou des mots de passe. Les secrets peuvent

être attaché aux pods lors de l'exécution afin que les données de configuration sensibles puissent être stockées en toute sécurité dans le cluster.

Il sont attachés au Pod comme un fichier ou variable d'environnement

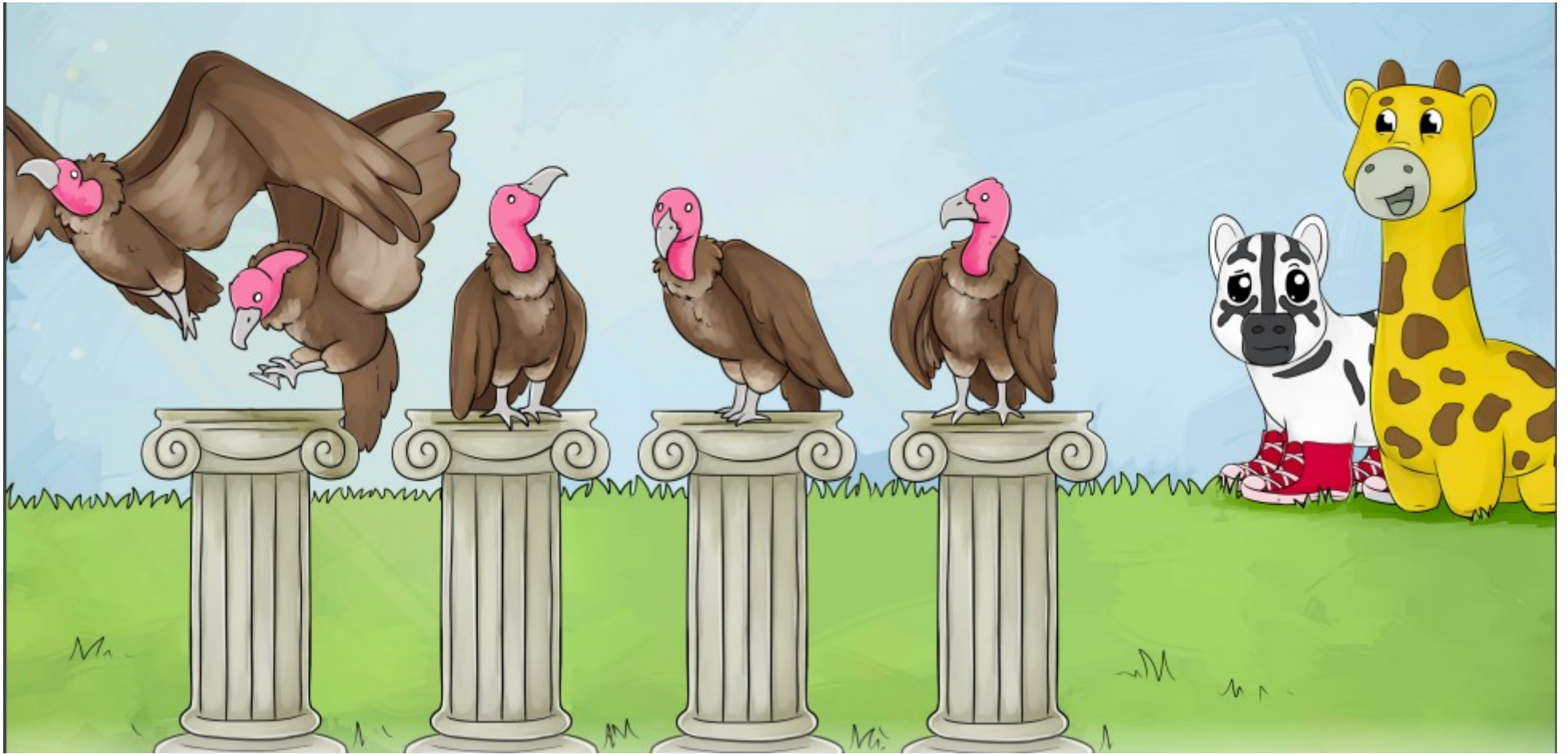
illustration



DaemonSet

Les DaemonSets permettent de s'assurer qu'une copie d'un pod est en cours d'exécution sur chaque nœud du cluster. Comme un cluster grandit et rétrécit, le DaemonSet répartit ces pods spécialement étiquetés sur tous les nœuds.

ilustration



- Services

Le service est la moyen d'accéder a un pod , un service K8s crée un point d'accès unique pour un groupe de pods.

Un service fournit une adresse IP et un port cohérents pour accéder aux pods sous-jacents.

Les utilisateurs externes et les pods internes utilisent les services pour communiquer avec d'autres pods

Architecture réseau des services

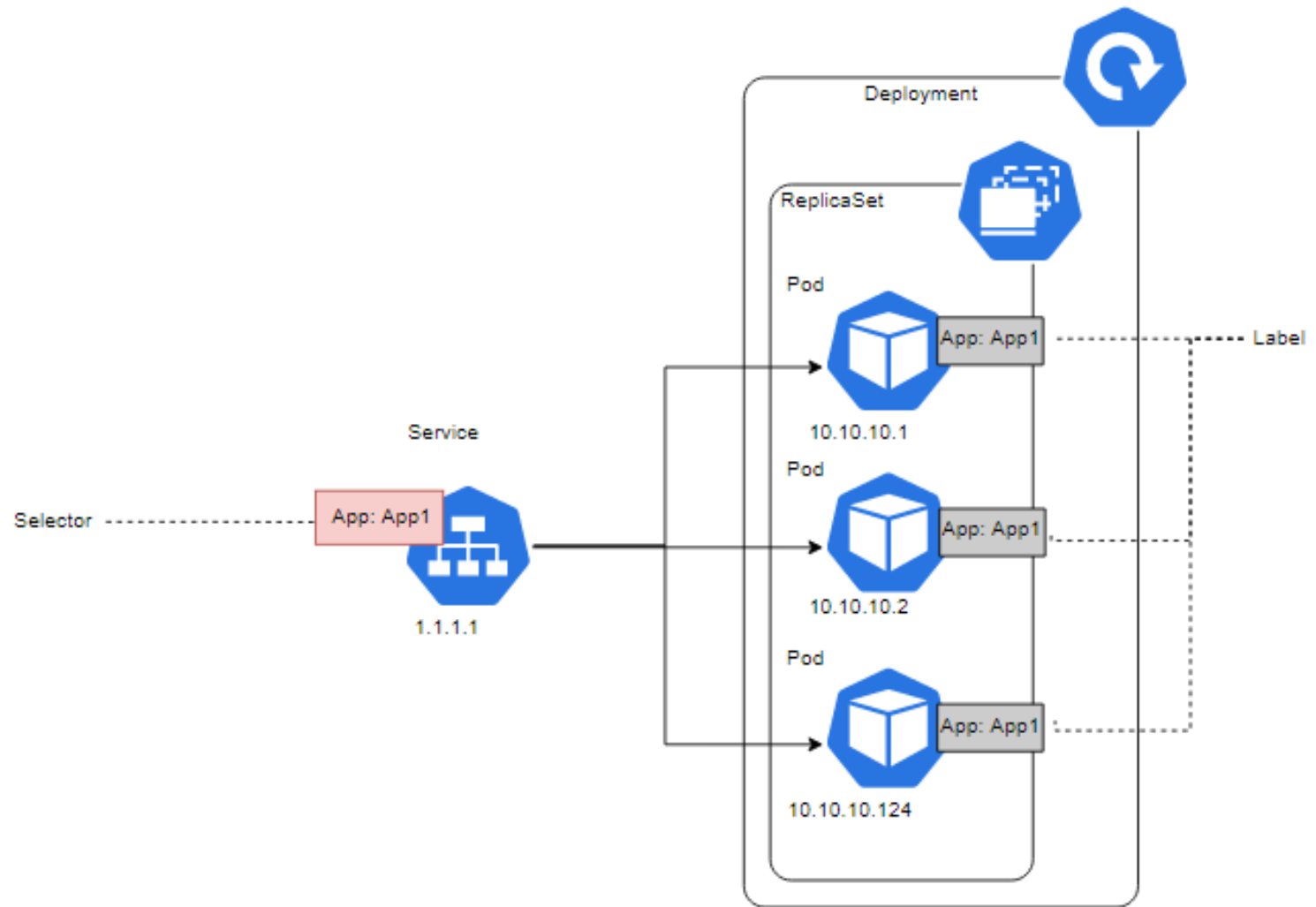
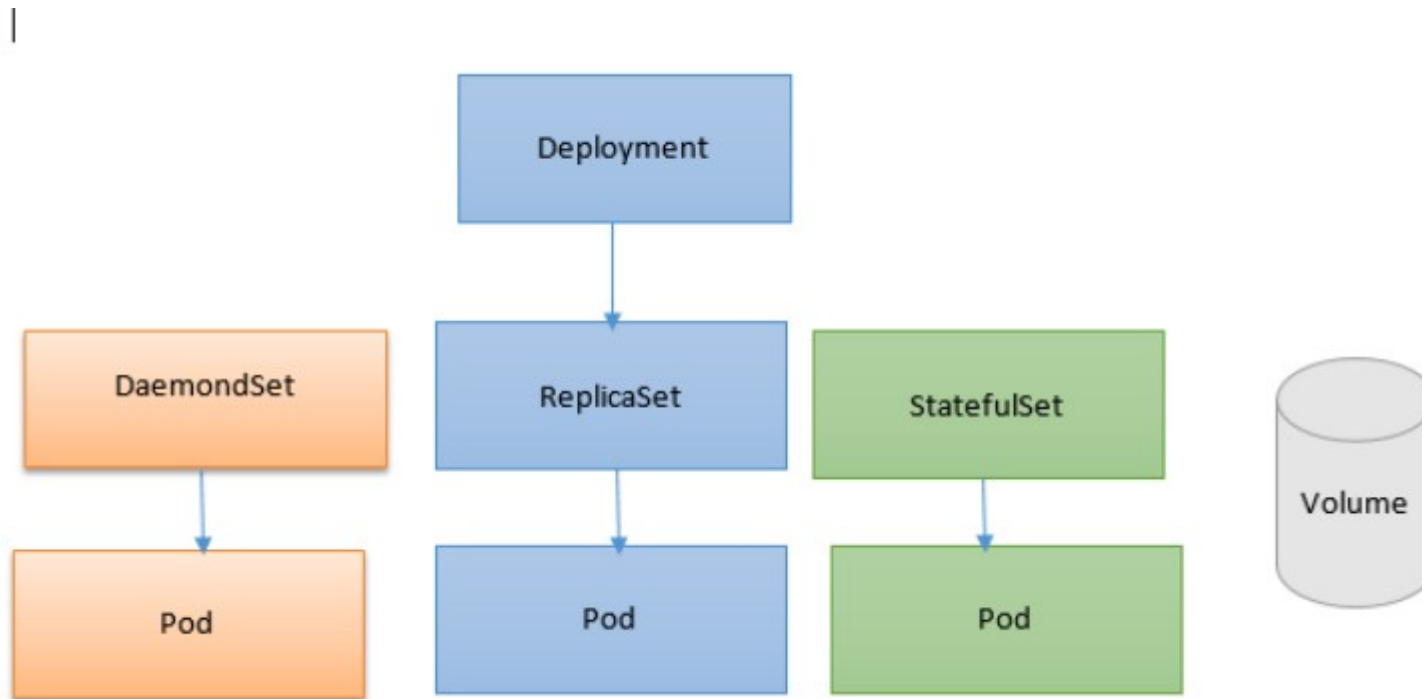


Fig : Niveau d'abstraction Deployment, DaemonSet, ReplicaSet, StatefulSet

- API server



Deployment.yaml

- apiVersion: apps/v1
- kind: Deployment
- metadata:
- name: nginx-deployment
- labels:
- app: nginx
- spec:
- replicas: 3
- selector:
- matchLabels:
- app: nginx
- template:
- metadata:
- labels:
- app: nginx

Explication des champ

- Le champ **apiVersion** est le premier champ obligatoire, et il spécifie le point de terminaison API sur le serveur API auquel nous voulons nous connecter ; il doit correspondre à une version existante pour le type d'objet défini.
- Le deuxième champ obligatoire est **kind**, spécifiant le type d'objet - dans notre cas, il s'agit de Deployment, mais il peut s'agir de Pod, Replicaset, Namespace, Service, etc.
- Le troisième champ obligatoire **metadata** contient les informations de base de l'objet, telles que le nom, les étiquettes , espace de noms, etc. Notre exemple montre deux champs spec (spec et spec.template.spec).
- La quatrième **spec** de champ obligatoire marque le début du bloc définissant l'état souhaité de l'objet Déploiement. Dans notre exemple, nous demandons que 3 **répliques**, ou 3 instances du pod, s'exécutent à un moment donné.
- Les pods sont créés à l'aide du modèle de **pod** défini dans **spec.template**. Un objet imbriqué, tel que le pod faisant partie d'un déploiement, conserve ses métadonnées et ses spécifications et perd l'apiVersion et le genre - tous deux étant remplacés par le modèle. Dans spec.template.spec, nous définissons l'état souhaité du Pod. Notre pod crée un seul conteneur exécutant l'image nginx:1.15.11 à partir de Hub Docker.

- Une fois l'objet de déploiement créé, le système Kubernetes attache le

► Fichier pod.yaml

- apiVersion: v1
- kind: Pod
- metadata:
 - name: nginx-pod
 - labels:
 - app: nginx
- spec:
 - containers:
 - - name: nginx
 - image: nginx:1.15.11
 - ports:
 - - containerPort: 80

Explication des champs

Le champ **apiVersion** doit spécifier v1 pour la définition de l'objet Pod.

- Le deuxième champ obligatoire est **kind** spécifiant le type d'objet Pod.
- La troisième **metadata** de champ obligatoire contient le nom et l'étiquette de l'objet.
- La quatrième **spec** de champ obligatoire marque le début du bloc définissant l'état souhaité de l'objet Pod - également appelé PodSpec. Notre pod crée un seul conteneur exécutant l'image nginx:1.15.11 de Docker Hub. Le champ containerPort spécifie le port de conteneur à exposer par les ressources Kubernetes pour l'accès inter-application ou l'accès client externe.

Service.yaml

- piVersion: v1
- kind: Service
- metadata:
 - name: nginx-service
- spec:
 - type: NodePort
 - ports:
 - - port: 80
 - targetPort: 5000
 - selector:

Explication des champ

- Dans cet exemple, nous créons un service **nginx-service** en sélectionnant tous les pods dont le Label **key=app** est défini sur **value=nginx**. Par défaut, chaque service reçoit une adresse IP routable uniquement à l'intérieur du cluster, appelée ClusterIP. Dans notre exemple, nous avons 172.17.0.4 et 172.17.0.5 comme ClusterIP assignés respectivement à nos services frontend-svc et db-svc.

MERCI !!!!!!!